ModKnapsack — an OpenModelica × Qiskit quantum knapsack toolchain

A proof-of-concept integration that drives the IBM Qiskit C API from an OpenModelica model through the Modelica external-C interface, and exercises the resulting solver on small 0/1 knapsack instances drawn from patent-portfolio analysis.

This repository accompanies the paper *Quantum Algorithms for Patent Portfolio Analysis Using OpenModelica and IBM Qiskit*, accepted for the 2026 North American Modelica & FMI Conference (Atlanta, October 12–14, 2026). The version of record is `ModKnapsack_Modelica2026-Paper.pdf` (also deposited on Zenodo, DOI 10.5281/zenodo.22034029). The paper's test cases stop at four items; the code in this repository solves instances of up to seven items.

Status: work in progress / proof of concept

The purpose of this project is to demonstrate the **integration** of a quantum software stack (Qiskit) with an equation-based modeling environment (OpenModelica) — not to present a new quantum algorithm, a scientific discovery, or a finished, production-ready framework. It makes **no claim of quantum advantage**: on every instance tried, the quantum search needs more oracle calls than brute-force enumeration (the advantage is asymptotic). The toolchain is under active development and its interfaces may change.

Contents

- `ModKnapsack.mo` — the OpenModelica package: a `TestCase` record, a `TestCases` library of instances, the external-C function `ModCallKnapsack`, and the model `KnapsackRun` that you simulate
- `knapsack/` — the C solver (`knapsack.c`, one file) and its nine self-test programs
- `CMakeLists.txt`, `run_knapsack.mos` — build configuration and the one-line run script
- `verify_qasm.py`, `noise_study.py`, `plot_noise.py`, `draw_circuit.py`, `hardware_run.py` — optional out-of-band Python tools that consume the exported OpenQASM-3 circuit (`hardware_run.py` is a demonstration of Qiskit Runtime submission; see *Solver paths* below)
- `ModKnapsack_Installation_Instructions.pdf` — build-from-scratch guide (Linux)
- `Knapsack_Documentation.pdf` — engineering documentation (algorithms, code structure, tests); its opening section lists what changed since it was written
- `ALGORITHM_REFERENCES.pdf` — literature map for the algorithms used
- `README.pdf` — this document

Qiskit itself is **not** bundled; you build its C API locally as described in the installation guide (validated against Qiskit commit `61e3ca016`). The buildable project sits at the repository root. `ModKnapsack.mo` locates `knapsack.c` and the Qiskit C API relative to its own location (`modelica://` URIs), so nothing in it needs editing after a clone; keep the layout the guide describes, with `qiskit/` inside the project root.

How the model calls the solver

`KnapsackRun` selects an instance from the `TestCases` library (twenty instances: the paper's four-item golden case, the two-item hardware case, and three families TC1 loose budget / TC2 tight budget / TC3 unique optimum at N = 2 … 7), lets you override the budget `MaxCost` and the `SolverPath`, and calls the C entry point once at initialisation (`stopTime = 0`):

```
void callKnapsack(int Npatents, const double *Values, const double *Costs, double MaxCost,
                  int *PatentIndices, int *ExStatus, int QDrawDetail, int SolverPath,
                  int NStates, double *PSubset, int *OracleCalls, int *Shots, int *Kind);
```

Every output is passed by pointer and the function returns `void` (OpenModelica passes scalar outputs by value if a C return value is also mapped). Outputs:

Output	Meaning
`PatentIndices[i]`	1 if item *i* is in the returned subset, else 0
`ExStatus`	0 optimum found · 1 verification run (paths 1/2) · −1 no feasible subset · −2 invalid input · −3 N > 7 · −6 external solver timed out
`PSubset[s]`	probability of the subset with bitmask *s* (`NStates = 2^N` entries, sums to 1)
`OracleCalls`	path 0: oracle calls spent by the search; 0 otherwise
`Shots`	paths 1/2: shots measured; 0 otherwise
`Kind`	0 = exact distribution of the final circuit (path 0), 1 = verification shots (paths 1/2)

The model computes `ActualCost` and `ActualValue` from `PatentIndices`, prints the instance, the result, and the most probable subsets, and compares the returned subset's probability with the blind-draw chance `1/2^N`. Costs and values are small integers (fractional inputs are rounded). `QDrawDetail` 0–5 selects an optional circuit drawing or OpenQASM-3 export of one oracle + diffuser iteration.

Solver paths

- **`SolverPath = 0` — statevector simulation (the default).** A pure-C statevector simulator built into `knapsack.c` runs the Grover search (Boyer–Brassard–Høyer–Tapp iteration schedule inside a Dürr–Høyer maximum search), best of `GROVER_K = 3` quantum

solves, with a classical check of the returned subset. `PSubset` is the exact output distribution of the final circuit. Up to `KNAPSACK_NMAX = 7` items (21 qubits).

- **`SolverPath = 1` (Aer noise model) and `2` (IBM QPU) — out-of-process verification runs.** The solver exports the *verification circuit* (one Grover search at the known optimum value, at the Grover-optimal iteration count) as `solverCircuit.qasm` plus a sidecar `solverCircuit.meta`, then waits for a results file — `aer_results.json` or `qpu_results.json` in the working directory — polling every 2 s for up to 2 h (`KNAPSACK_WAIT_TIMEOUT_S` overrides the limit). **This repository does not include the program that runs the circuit and writes that file**; without one, paths 1 and 2 end with `ExStatus = −6` after the timeout. A runner must execute the exported circuit and write, atomically (write to a temporary name, then rename), a JSON object with these keys:

key	content
`request_id`	the integer from `solverCircuit.meta` (must match this request)
`path`	1 or 2, as in the sidecar
`shots`	number of shots
`counts_by_mask`	array of `2^N` integers, index = item bitmask (bit i = item i), summing to `shots`
`backend`, `job_id`	strings, reported only

The sidecar lists `N`, `NQ`, `W`, `T` (threshold), `j` (iterations), `path`, `request_id`, the `item_qubits` to measure and the `marked_masks` (the optimal subsets). The solver validates the results file against the request, takes the most frequent feasible measured subset as the answer, returns the measured distribution (share of shots per subset) as `PSubset`, and flags the run with `ExStatus = 1` / `Kind = 1`.

Build and test

Full instructions, from a bare Ubuntu machine to the end-to-end OpenModelica run, are in `ModKnapsack_Installation_Instructions.pdf`. In short, with the Qiskit C API built under `qiskit/dist/c/`:

```
cmake -S . -B cmake-build-debug && cmake --build cmake-build-debug
export LD_LIBRARY_PATH=$PWD/qiskit/dist/c/lib:$LD_LIBRARY_PATH
for t in gate_test adder_test cmp_test oracle_test diffuser_test solver_test qasm_test draw_
  ./cmake-build-debug/$t || echo "$t FAILED"; done
omc run_knapsack.mos        # golden case: items 0 and 2, cost 6, value 8, ExStatus 0
```

`knapsack_test` exercises `callKnapsack()` itself on seven cases and exits non-zero on any failure; `solver_test` runs full searches (a couple of minutes). To run another library instance from a script or OMEdit:

```
model Run extends ModKnapsack.KnapsackRun(tc = ModKnapsack.TestCases.TC3_N5, MaxCost = 4); e
```

Disclaimer

The code and models in this repository are released under the MIT License (see `LICENSE`) and are provided strictly **"as is"**, without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and non-infringement. The author makes no representation regarding the accuracy, completeness, or suitability of this software for any purpose, and assumes **no liability or accountability** for any errors or omissions, or for any damages or system failures arising from its use. Use it at your own risk.

Use of AI tools

During the preparation of this work the author used Anthropic's Claude (via Claude Code) as a development and editing assistant. All algorithmic design, code, mathematical formulations and text were directed, independently reviewed and verified by the author, who retains sole intellectual property ownership of, and full responsibility for, the final work. The same statement appears in the header of every source file and in the paper's Acknowledgment.

Third-party notices

- **Qiskit** (IBM, Apache License 2.0) is a build-time dependency. It is not bundled; you clone it and build the C API locally as the installation guide describes, and `qiskit/` is git-ignored.
- The `mcx` (C3X) gate definition that `write_qasm` emits in `knapsack/knapsack.c` is reproduced from Qiskit's OpenQASM 2 standard library (`qiskit/qasm/libs/qelib1.inc`, `gate c3x`), Copyright 2017 IBM and its contributors, Apache License 2.0 (https://www.apache.org/licenses/LICENSE-2.0), with the qubit parameters renamed.
- All other code in this repository is original work released under the MIT License below.

License

MIT License — Copyright (c) 2026 Mark Myers. See `LICENSE`.